

Методичка 5. Списки в Python

1. Зачем нужна эта тема

Список нужен, когда в программе нужно хранить много значений.

Если нужно сохранить одно число, достаточно переменной:

```
age = 15
```

Но если нужно сохранить оценки, имена, цены товаров или результаты игроков, отдельных переменных становится слишком много:

```
mark1 = 5
mark2 = 4
mark3 = 3
mark4 = 5
```

Такой код неудобно читать и обрабатывать.

Список позволяет хранить много значений в одной переменной:

```
marks = [5, 4, 3, 5]
```

Списки используются почти во всех программах:

- список оценок;
- список игроков;
- список товаров;
- список вопросов;
- список ответов;
- список чисел;
- список команд;
- список объектов.

Эта тема связана с циклами. Список часто обрабатывается через `for` или `while`.

2. Главная идея темы

Список — это коллекция значений, которые идут по порядку.

У каждого элемента списка есть номер. Этот номер называется индексом.

Важно: индексы в Python начинаются с 0.

```
numbers = [10, 20, 30]
```

Здесь:

- `numbers[0]` — это `10` ;
- `numbers[1]` — это `20` ;
- `numbers[2]` — это `30` .

Список — структура данных для хранения нескольких значений.

Элемент списка — одно значение внутри списка.

Индекс — номер элемента в списке.

Длина списка — количество элементов в списке.

3. Базовый синтаксис

Создание списка

```
numbers = [10, 20, 30]
names = ["Аня", "Борис", "Вика"]
```

Список записывается в квадратных скобках.

Получение элемента по индексу

```
numbers = [10, 20, 30]

print(numbers[0])
print(numbers[1])
print(numbers[2])
```

Результат:

```
10  
20  
30
```

На что обратить внимание:

- первый элемент имеет индекс 0;
- последний элемент в этом списке имеет индекс 2.

Изменение элемента

```
numbers = [10, 20, 30]  
  
numbers[1] = 99  
  
print(numbers)
```

Результат:

```
[10, 99, 30]
```

Списки можно изменять.

Добавление элемента

```
numbers = []  
  
numbers.append(10)  
numbers.append(20)  
  
print(numbers)
```

Результат:

```
[10, 20]
```

Метод `append()` добавляет элемент в конец списка.

Вставка элемента

```
numbers = [10, 30]

numbers.insert(1, 20)

print(numbers)
```

Результат:

```
[10, 20, 30]
```

`insert(1, 20)` означает: вставить число `20` на индекс `1`.

Удаление элемента

```
numbers = [10, 20, 30]

numbers.pop(1)

print(numbers)
```

Результат:

```
[10, 30]
```

`pop(1)` удаляет элемент с индексом 1.

Длина списка

```
numbers = [10, 20, 30]

print(len(numbers))
```

Результат:

```
3
```

`len()` показывает количество элементов.

4. Разбор темы от простого к сложному

Шаг 1. Хранение нескольких значений

```
marks = [5, 4, 3, 5]

print(marks)
```

Список позволяет хранить все оценки в одной переменной.

Шаг 2. Обращение к элементам

```
marks = [5, 4, 3, 5]

print("Первая оценка:", marks[0])
print("Вторая оценка:", marks[1])
```

Индекс помогает получить конкретный элемент.

Шаг 3. Изменение значения

```
marks = [5, 4, 3, 5]

marks[2] = 4

print(marks)
```

Третья оценка изменилась с 3 на 4.

Шаг 4. Добавление значений

```
marks = []

mark = int(input("Введите оценку: "))
marks.append(mark)

mark = int(input("Введите оценку: "))
marks.append(mark)

print(marks)
```

Так список можно заполнять постепенно.

Шаг 5. Заполнение списка через цикл

```
marks = []

for i in range(5):
    mark = int(input("Введите оценку: "))
    marks.append(mark)

print(marks)
```

Программа вводит 5 оценок и сохраняет их в список.

Шаг 6. Перебор списка по значениям

```
marks = [5, 4, 3, 5]

for mark in marks:
    print(mark)
```

Такой вариант удобен, если нужно просто посмотреть каждый элемент.

Шаг 7. Перебор списка по индексам

```
marks = [5, 4, 3, 5]

for i in range(len(marks)):
    print("Индекс:", i, "Оценка:", marks[i])
```

Такой вариант нужен, когда важно знать индекс элемента.

Шаг 8. Поиск суммы элементов

```
numbers = [10, 20, 30]
total = 0

for number in numbers:
    total = total + number

print("Сумма:", total)
```

Список удобно обрабатывать через цикл.

Шаг 9. Поиск максимального элемента

```
numbers = [8, 3, 15, 6]

max_number = numbers[0]

for number in numbers:
    if number > max_number:
        max_number = number

print("Максимум:", max_number)
```

Программа постепенно сравнивает элементы и запоминает самый большой.

5. Частые ошибки учеников

Ошибка 1. Думают, что индексы начинаются с 1

Неправильно:

```
numbers = [10, 20, 30]

print(numbers[1])
```

Почему это ошибка:

Если нужно получить первый элемент, индекс должен быть 0. `numbers[1]` — это второй элемент.

Правильно:

```
numbers = [10, 20, 30]

print(numbers[0])
```

Ошибка 2. Выходят за границы списка

Неправильно:

```
numbers = [10, 20, 30]

print(numbers[3])
```

Почему это ошибка:

В списке 3 элемента, их индексы: 0, 1, 2. Индекса 3 нет.

Правильно:

```
numbers = [10, 20, 30]

print(numbers[2])
```

Ошибка 3. Путают индекс и значение

Неправильно:

```
numbers = [10, 20, 30]

for number in numbers:
    print(numbers[number])
```

Почему это ошибка:

`number` — это значение элемента, а не индекс. Первое значение равно 10, но индекса 10 в списке нет.

Правильно:

```
numbers = [10, 20, 30]

for number in numbers:
    print(number)
```

Или так:

```
numbers = [10, 20, 30]

for i in range(len(numbers)):
    print(numbers[i])
```

Ошибка 4. Забывают скобки у append

Неправильно:

```
numbers = []

numbers.append = 10
```

Почему это ошибка:

`append` — это метод. Его нужно вызывать через круглые скобки.

Правильно:

```
numbers = []

numbers.append(10)
```

Ошибка 5. Создают список внутри цикла

Неправильно:

```
for i in range(5):
    numbers = []
    number = int(input("Введите число: "))
    numbers.append(number)

print(numbers)
```

Почему это ошибка:

На каждом шаге создаётся новый пустой список. Старые значения теряются.

Правильно:

```
numbers = []

for i in range(5):
    number = int(input("Введите число: "))
    numbers.append(number)

print(numbers)
```

Ошибка 6. Удаляют элементы во время обычного перебора

Неправильно:

```
numbers = [1, 2, 3, 4, 5]

for number in numbers:
    if number % 2 == 0:
        numbers.remove(number)

print(numbers)
```

Почему это опасно:

Когда список изменяется во время перебора, некоторые элементы могут быть пропущены.

Лучше создать новый список:

```
numbers = [1, 2, 3, 4, 5]
result = []

for number in numbers:
    if number % 2 != 0:
        result.append(number)

print(result)
```

Ошибка 7. Используют много переменных вместо списка

Неправильно:

```
mark1 = 5
mark2 = 4
mark3 = 3

print(mark1)
print(mark2)
print(mark3)
```

Почему это неудобно:

Если оценок станет 30, код станет огромным.

Правильно:

```
marks = [5, 4, 3]

for mark in marks:
    print(mark)
```

6. Мини-примеры для закрепления

Пример 1. Список чисел

```
numbers = [1, 2, 3, 4, 5]

print(numbers)
```

Программа хранит несколько чисел в одной переменной. Закрепляется создание списка.

Пример 2. Первый и последний элемент

```
numbers = [10, 20, 30, 40]

print(numbers[0])
print(numbers[3])
```

Закрепляется обращение по индексу.

Пример 3. Добавление элементов

```
names = []

names.append("Аня")
names.append("Борис")

print(names)
```

Закрепляется метод `append` .

Пример 4. Заполнение списка

```
numbers = []

for i in range(3):
    number = int(input("Введите число: "))
    numbers.append(number)

print(numbers)
```

Закрепляется заполнение списка через цикл.

Пример 5. Сумма элементов

```
numbers = [4, 7, 2]
total = 0

for number in numbers:
    total = total + number

print(total)
```

Закрепляется перебор списка и накопитель.

Пример 6. Количество положительных чисел

```
numbers = [-2, 5, 0, 7, -1]
count = 0

for number in numbers:
    if number > 0:
        count = count + 1

print(count)
```

Закрепляется условие внутри цикла.

Пример 7. Изменение элемента

```
items = ["хлеб", "молоко", "сыр"]

items[1] = "кефир"

print(items)
```

Закрепляется изменение значения по индексу.

Пример 8. Перебор с индексами

```
names = ["Аня", "Борис", "Вика"]

for i in range(len(names)):
    print(i, names[i])
```

Закрепляется `len()` и перебор по индексам.

7. Практические задания

Лёгкий уровень

1. Создать список из 5 чисел и вывести его.
2. Создать список из 3 имён и вывести каждое имя на новой строке.
3. Дан список чисел. Вывести первый элемент.
4. Дан список чисел. Вывести последний элемент.
5. Дан список оценок. Изменить вторую оценку на 5.

Средний уровень

1. Пользователь вводит 5 чисел. Сохранить их в список и вывести список.
2. Дан список чисел. Посчитать сумму всех элементов.
3. Дан список чисел. Посчитать количество чётных чисел.
4. Дан список чисел. Найти максимальный элемент.
5. Дан список строк. Вывести только те строки, длина которых больше 5.

Повышенный уровень

1. Пользователь вводит 10 чисел. Создать новый список только из положительных чисел.
2. Дан список чисел. Создать новый список, где каждый элемент увеличен в 2 раза.
3. Дан список оценок. Посчитать среднюю оценку.

8. Контрольные вопросы

1. Что такое список?
2. Для чего нужны списки?
3. Что такое элемент списка?
4. Что такое индекс?
5. С какого числа начинаются индексы в Python?
6. Что делает `append()` ?
7. Что делает `insert()` ?
8. Что делает `pop()` ?
9. Что показывает `len()` ?
10. Чем отличается перебор по значениям от перебора по индексам?
11. Почему нельзя обращаться к индексу, которого нет?
12. Почему список лучше, чем много отдельных переменных?

9. Краткий конспект в конце

Главная идея темы: список хранит много значений в одной переменной.

Основные конструкции:

```
numbers = [10, 20, 30]
```

```
print(numbers[0])
```

```
numbers.append(40)
```

```
numbers[1] = 99
```

```
for number in numbers:  
    print(number)
```

```
for i in range(len(numbers)):  
    print(numbers[i])
```

Важно запомнить:

- индексы начинаются с 0;
- список можно изменять;
- `append()` добавляет элемент в конец;
- `insert()` вставляет элемент;
- `pop()` удаляет элемент;
- `len()` показывает длину списка;
- список удобно обрабатывать через цикл.

Частые ошибки:

- путать индекс и значение;
- выходить за границы списка;
- забывать скобки у `append`;
- создавать список внутри цикла;
- изменять список во время обычного перебора.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- создавать списки;
- получать элементы по индексу;
- изменять элементы списка;
- добавлять элементы в список;
- удалять элементы из списка;
- находить длину списка;
- перебирать список через цикл;
- считать сумму элементов;
- искать количество подходящих элементов;
- решать простые задачи на коллекции.

